Instrument Droid (Board 4)

1. OBJECTIVES

- Design a Instrument Droid consisting of 4 Layers in Altium using PCB best practices to measure the Thevenin's resistance of a power source up to 12V using ATMEGA328P.

2. COMPONENTS USED/BOM

1. BZ1 - Buzzers Transducer - 1
2. C1, C9, C10, C11, C12, C13, C14, C15, C16, C17, C18, C19, C20, C21 - 22uF Ceramic - 14
3. C3, C4, C5, C6 - 22pF Ceramic - 4
4. C7 - 1uF Ceramic - 1
5. C8 - 47nF Ceramic - 1
6. D1 - TVS Diode - 1
7. D2, D3, D4, D5 - RGB LED - 4
8. J1 - 6-position Header - 1
9. J2 - USB Mini - 1
10. J3 - Female Header - 1
11. L1 - 10uH Inductor - 1
12. LED1, LED2, LED4, LED5, LED6 - Red LED - 5
13. P1, P3 - Power Jack - 2
14. P2 - Screw Terminal - 1
15. Q1 - N-Channel MOSFET - 1
16. R1, R2 - 1M Resistor - 2
17. R3 - 500m Shunt - 1
18. R4, R5, R8, R9, R10, R11, R14 - 1k Resistor - 7
19. R6 - 300 Ohm - 1
20. R7, R12, R13, R15, R18, R19 - 10k Resistor - 6
21. R16 - 1 Ohm - 1
22. R17 - 10 Ohm - 1
23. R20, R21 - 0 Ohm - 2
24. SW1, SW4, SW5 - 3-Pin Header - 3
25. SW2 - 2-Pin Header - 1
26. SW3 - Tactile Switch - 1
27. U1 - ATMEGA328P Microcontroller - 1
28. U2 - CH340G Transceiver - 1
29. U3 - DAC IC - 1
30. U4 - Operational Amplifier - 1
31. U5 - ADS1115IDGSR ADC - 1
32. Y1 - 16MHz Crystal - 1
33. Y2 - 12MHz Crystal - 1

3. PLAN OF RECORD (POR)

The PCB circuit should contain the following

- **DC Input:** A barrel jack/connector for an AC-to-5V DC adapter or USB connector to power the board.
- **DUT Connectors**: Input interfaces for measuring the Thevenin's Resistance of a power supply
- **Display**: An OLED screen dedicated to displaying the measured Thevenin's Resistance values.
- **UI/Aesthetics**: Four smart LEDs and a buzzer for user feedback and visual effects.
- **PC Interface**: USB Mini-B connector featuring a TVS diode for transient protection.
- **Measuring circuitry**: Circuitry comprising a MOSFET, Operational Amplifier (Op-Amp), ADC, and DAC.
- **Power Select**: A toggle switch to choose between external 5V DC or USB power.
- **Programming Interface**: ICSP header pins for bootloading the ATmega328.
- **Logic Hardware**: ATmega328 microcontroller and CH340g chip for USB-to-UART communication.
- **Inrush Measurement**: A 500mohm sense resistor accompanied by test points for inrush current analysis.
- **System Control**: Onboard reset circuitry
- **Noise Filtering**: Ferrite filter applied to the AVCC (Analog Voltage) pin.
- **Timing**: 16 MHz crystal oscillator for the ATmega328 and 12 MHz for the CH340g.
- **Status Indicators**: LEDs to indicate status for the Main Power Supply and VRM
- **Debug Points**: Test points accessible for the 5V rail, I2C bus, UART, and USB data lines (D+, D-)
- **Isolation**: Header pins provided for isolating the ATmega328 pins
- **Load Selection**: A switch to select the sense resistor value (1 or 10 ohms) located at the source of the MOSFET.

4. ROUGH SKETCH AND CIRCUIT IMAGE

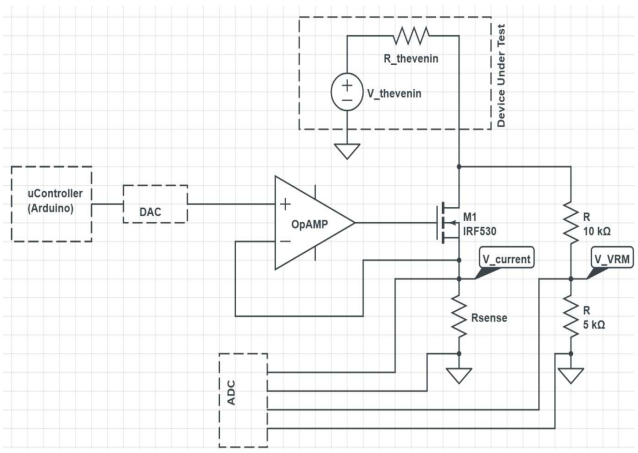


Fig 1 - Rough Sketch

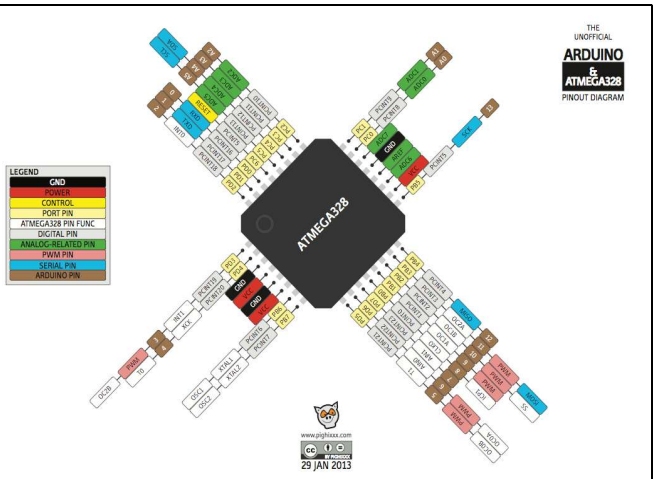
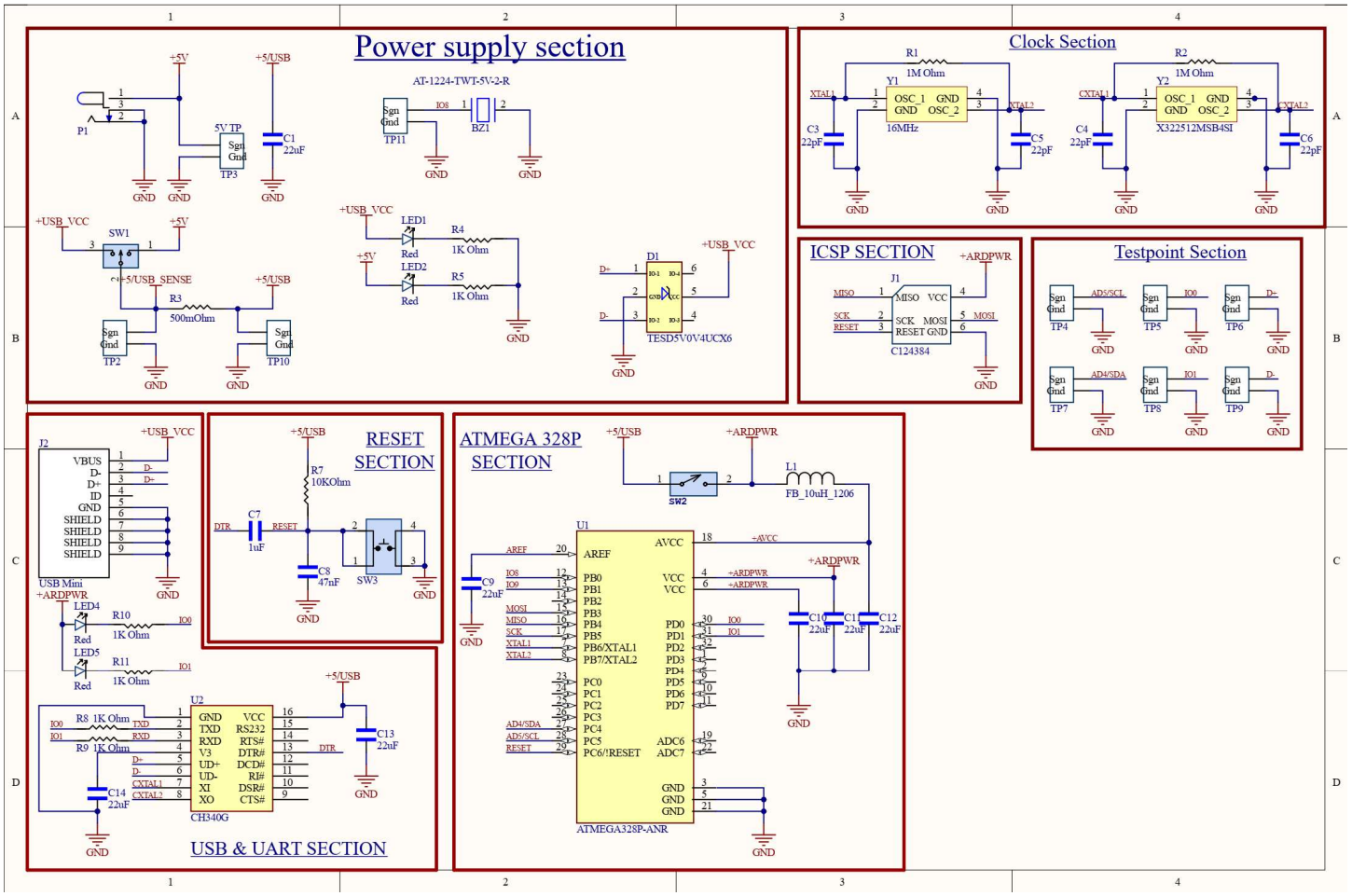


Fig 2 – Atmega328 Pinouts

5. SCHEMATIC DIAGRAM



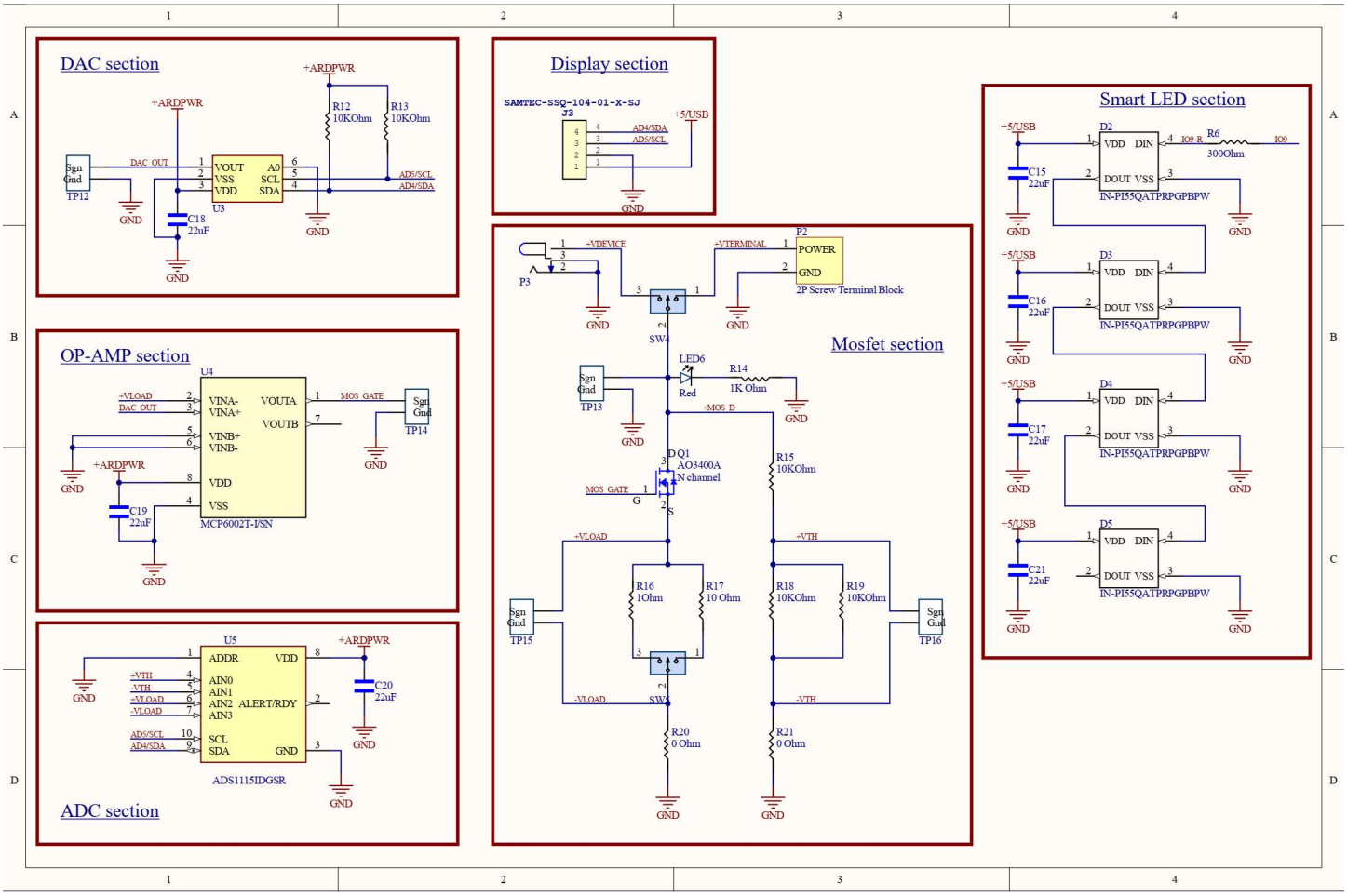
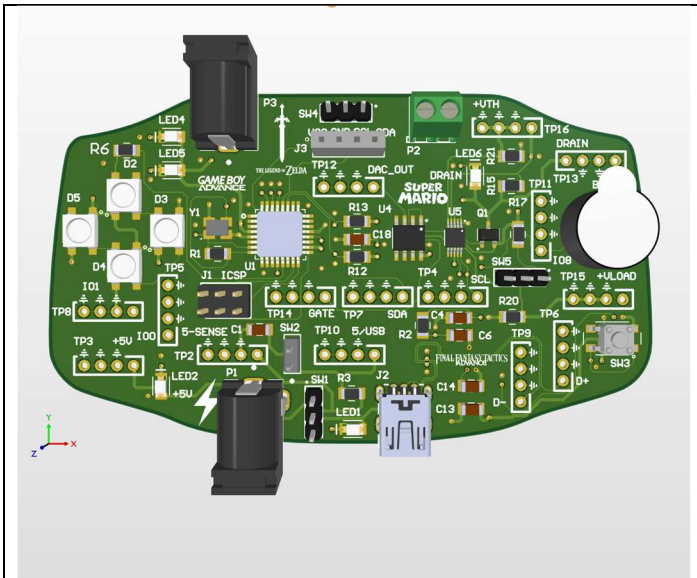


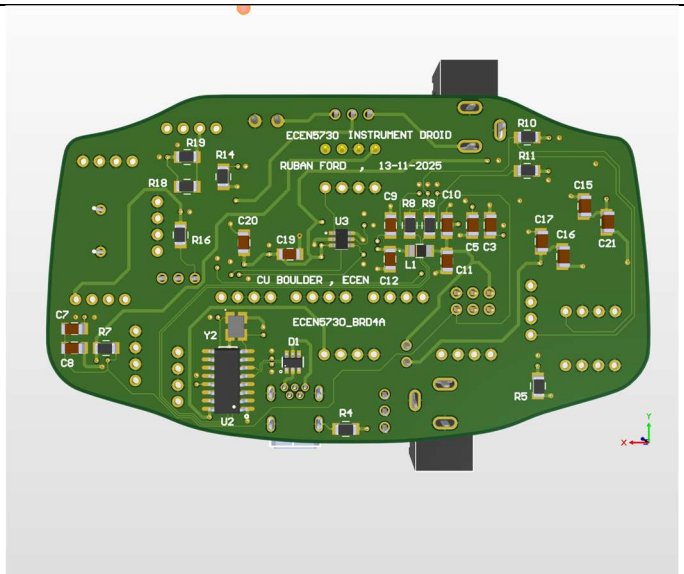
Figure 3 - Schematic diagram in Altium software

6. 3D Layout & PCB Layout

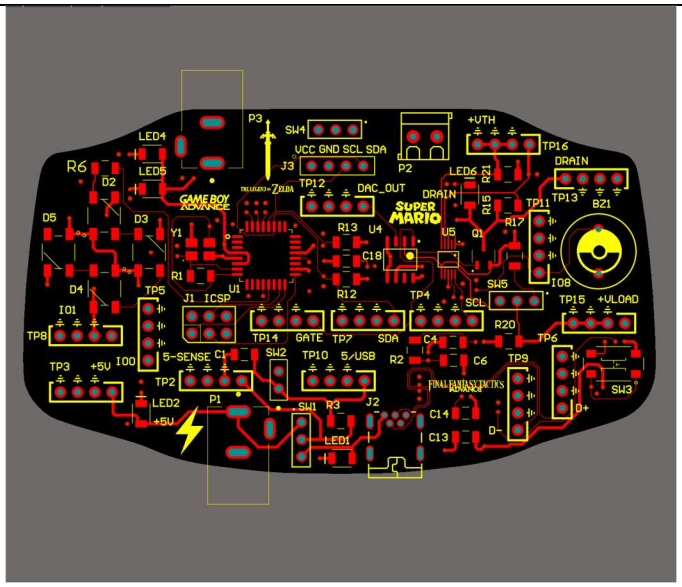
The Board outline and Layout is inspired from Gameboy Advance.



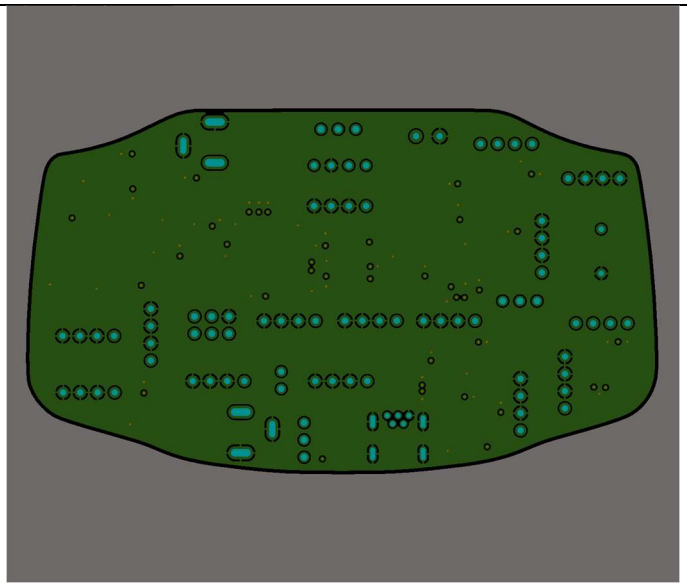
3D - Compside

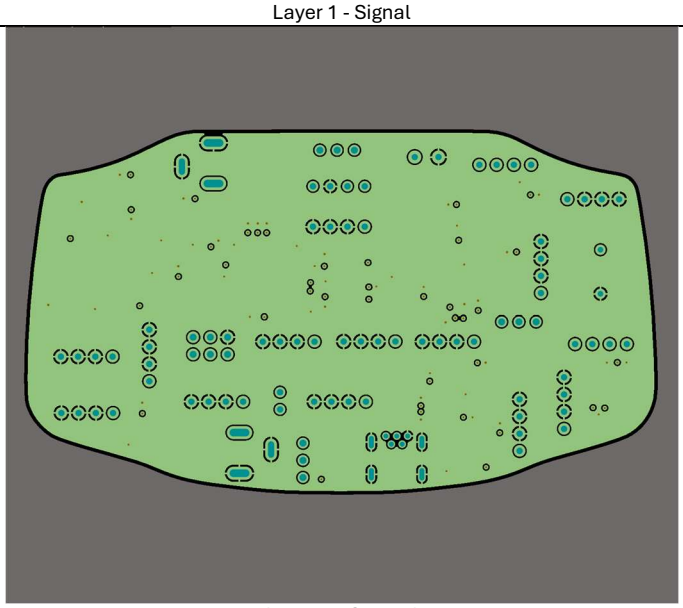


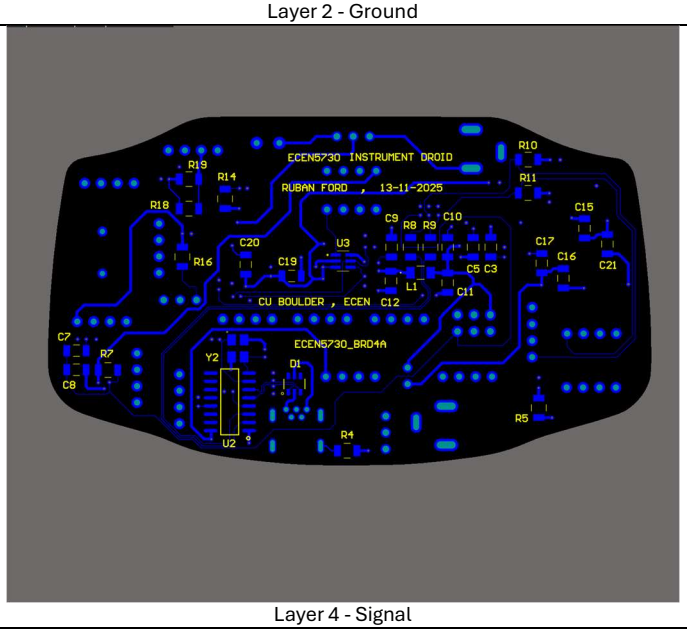
3D - Soldside



Layer 1 - Signal


Layer 2 - Ground


Layer 3 - Ground



Layer 4 - Signal

Figure 4 - PCB Layout in Altium software

7. ASSEMBLY PROCESS

The PCB was populated using a surface-mount assembly process, which included solder paste application via stencil, manual pick-and-place component mounting, and convection reflow soldering.

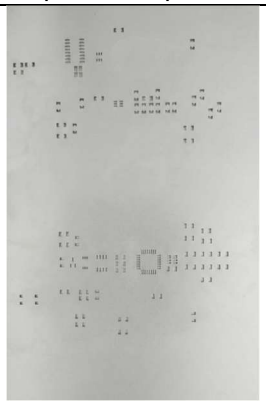


Fig 5: Stencil Cutout

Fig 6: manual pick-and-place component mounting

8. BEFORE AND AFTER MANUAL ASSEMBLY

- **BEFORE MANUAL ASSEMBLY**

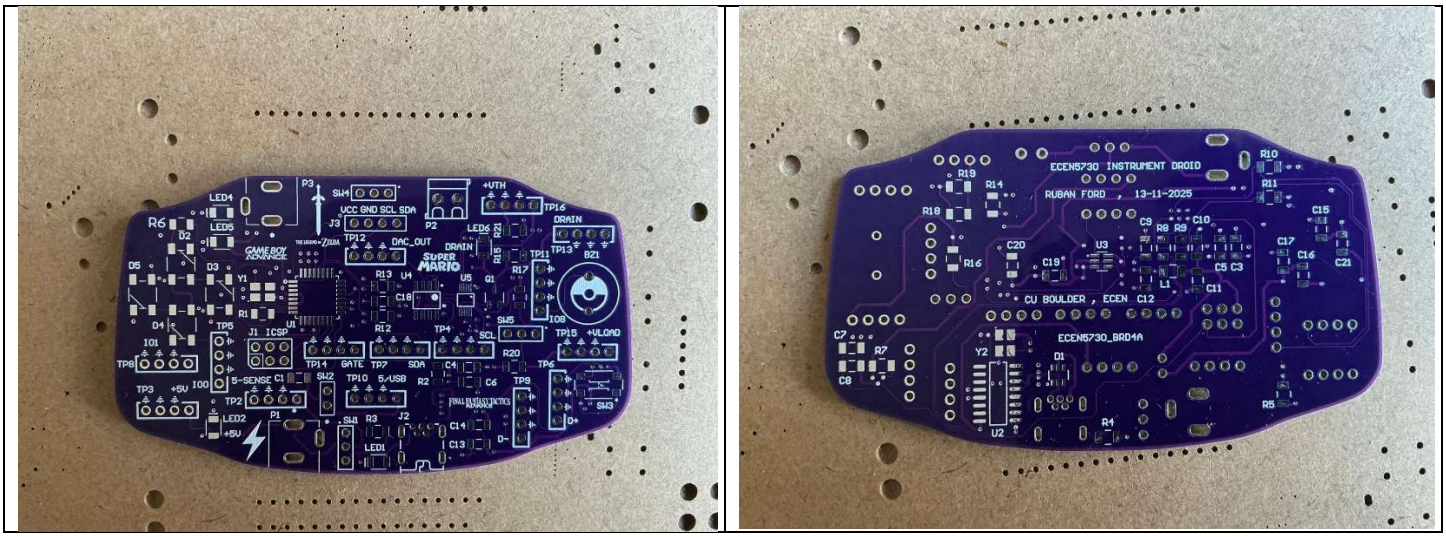


Figure 7 – Before Assembly (Left → Compside ; Right → Soldside)

- **AFTER MANUAL ASSEMBLY**

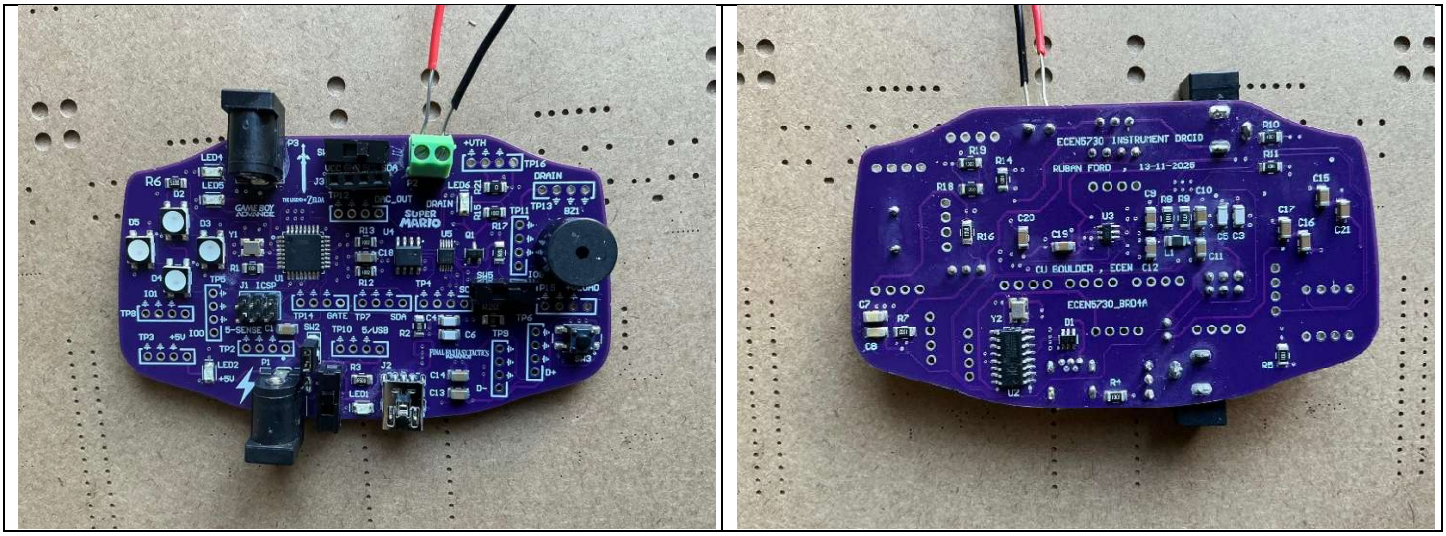
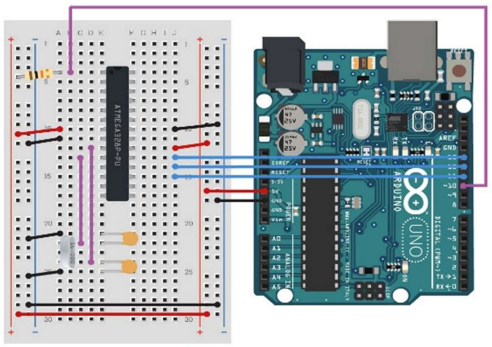


Figure 8 - After assembly (Left → Compside ; Right → Soldside)

9. WORKING CRITERIA

- Indicator LED1 or LED2 should turn ON immediately when the board is powered up.
- Indicator LED6 should turn ON specifically when the VRM (Voltage Regulator Module) is connected to the board.
- Test Point 10 should show 5V when the board is powered by either the AC-to-DC adapter or the USB input.
- The ATmega328 should accept the bootloader via the ICSP/SPI headers when connected to a commercial Arduino Uno acting as an ISP.
- The 16MHz & 12 MHz crystal should resonate at the correct frequency after bootloading, allowing the microcontroller to execute code uploaded from the Arduino IDE.
- The board should measure the Thevenin's Resistance of any external power supply (up to 12V) once the appropriate measurement code is uploaded.

	1. Connect SCK to SCK 2. MISO to MISO 3. MOSI to MOSI 4. VCC to 5V header socket (of new board) and gnd to gnd (of new board) 5. Pin 10 to Reset header pin (of new board) 6. After this is done, connect commercial Arduino to PC and go to File> examples>11(Arduino ISP), upload it. 7. Tools>programmer (Arduino as ISP) and make sure port and board is selected appropriately. Then click Burn bootloader.
Fig 9 : Boot loading Atmega328	Fig 10 : Steps for bootloading

10. AFTER POWERING UP

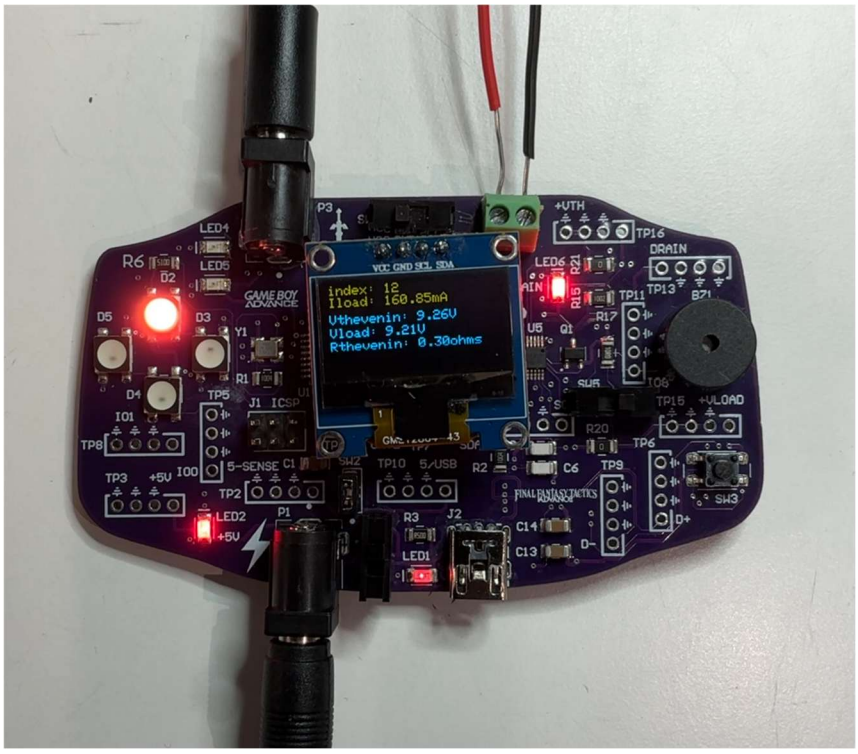


Figure 11 - Board after powering up

- The 5V indication LED lit up showing there was no issues with the power supply. Measurements were also taken using TP10 to make sure of the same. USB LED also lit up when powered through the USB cable.
- Atmega328 was boot loaded using External Arduino and simple melody code was uploaded to check its functionality.
- The code to measure Thevenin's resistance was uploaded and OLED display and serial monitor was used to see the Thevenin's resistance. The LED indicator for VRM was lit up when Device under test was connected to VRM.
- You can view the full working of the board with **SMART LED's and Buzzer** in this link : https://drive.google.com/file/d/1OG99ApjhSHEhDRxTujz3uAynlOdEklqi/view?usp=drive_link

11. DEBUGGING

There was no Hard/Soft errors encountered and board functioned as expected.

12. SCOPE SHOT AND ANALYSIS

The board is powered by 5V DC adapter or USB cable which can be controlled by SW1. Measurements are taken in respective Test points and checked for any errors.

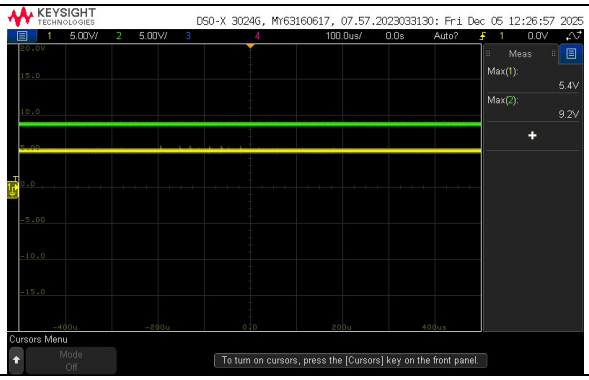


Fig 12 : 5V(yellow) DC Supply and 9V(green) device under test measured in TP10 & TP13

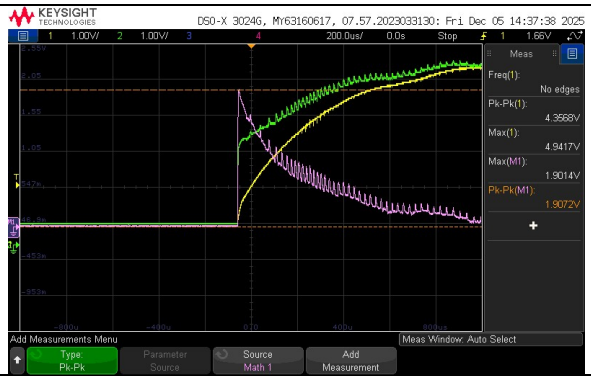


Fig 13 : inrush current measured through 500mohm sense resistor.
Green → High side of resistor
Yellow → Low side of resistor
Pink → Math function(Green-Yellow)
Inrush current = 1.9 volts/0.5ohms = **3.8A !**

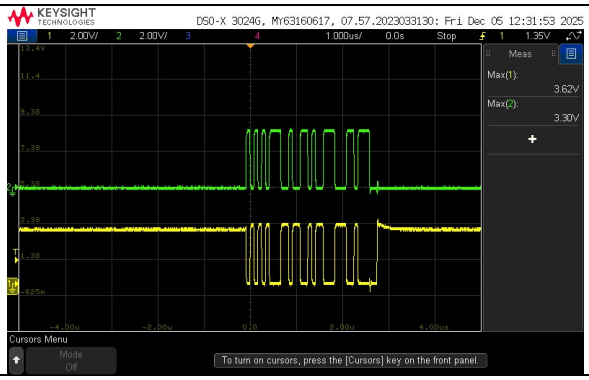


Fig 14 : USB data measured in TP6(yellow) & TP9(green) while uploading the code.

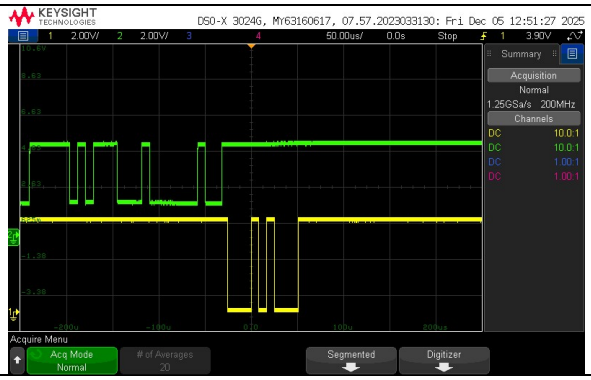


Fig 15 : UART signals measured in TP5(yellow) & TP8(green) while uploading the code.

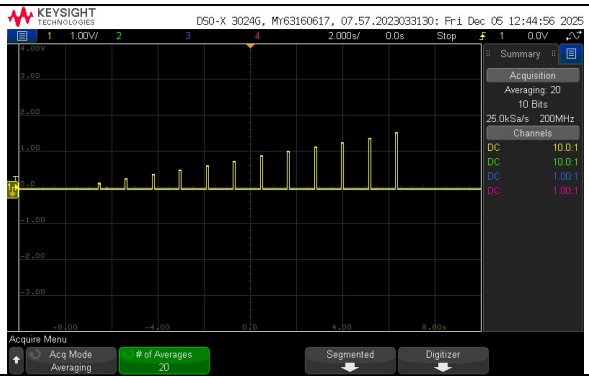


Fig 16 : DAC output measured in TP12

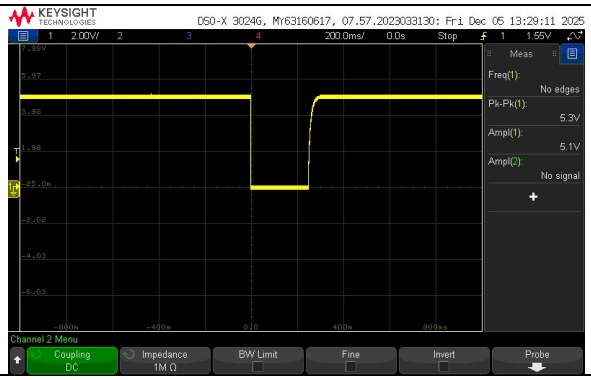


Fig 17 : Reset signal measured using normal trigger

Measuring Thevenin's Resistance by measuring the Voltage across the voltage divider and current through the sense resistor using testpoints TP16 and TP15 respectively.

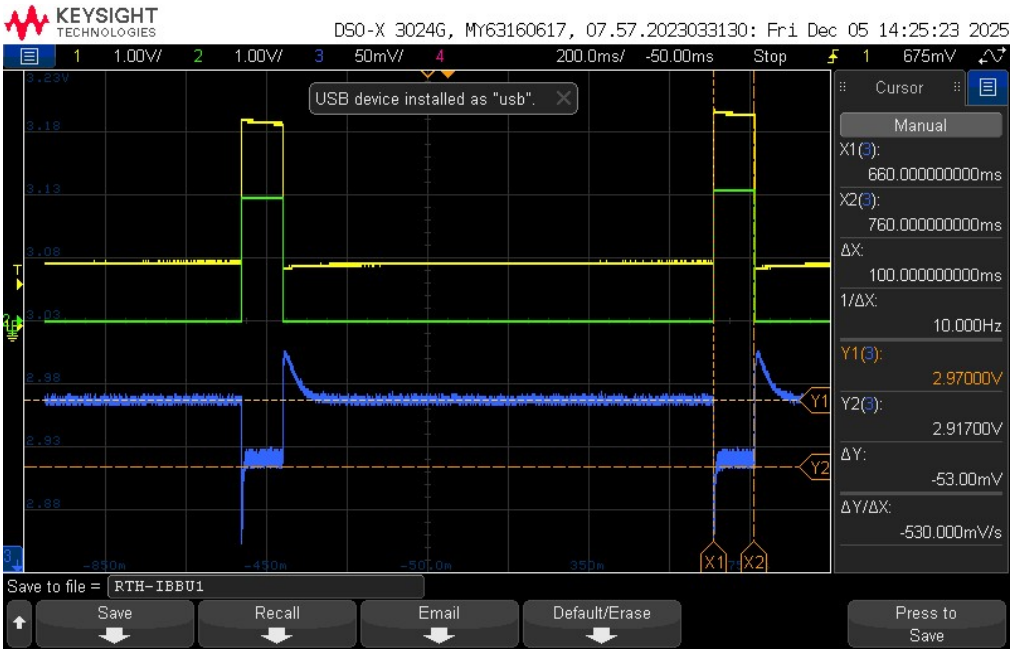


Fig 18 : Thevenin's resistance

Yellow → Gate signal ; Green → Voltage across sense resistor ; Blue → Voltage across voltage divider.

When a 9V wall supply is connected ,

The voltage before drop (Vth) = 2.97V, So total voltage = $2.97 * \frac{R1+R2}{R2} = 2.97 * \left(\frac{15000}{5000}\right) = 8.91V$

The voltage after drop (Vload) = 2.91V, So total voltage = $2.91 * \frac{R1+R2}{R2} = 2.91 * \left(\frac{15000}{5000}\right) = 8.73V$

Iload (Green) = 2V/10ohms = 0.2A

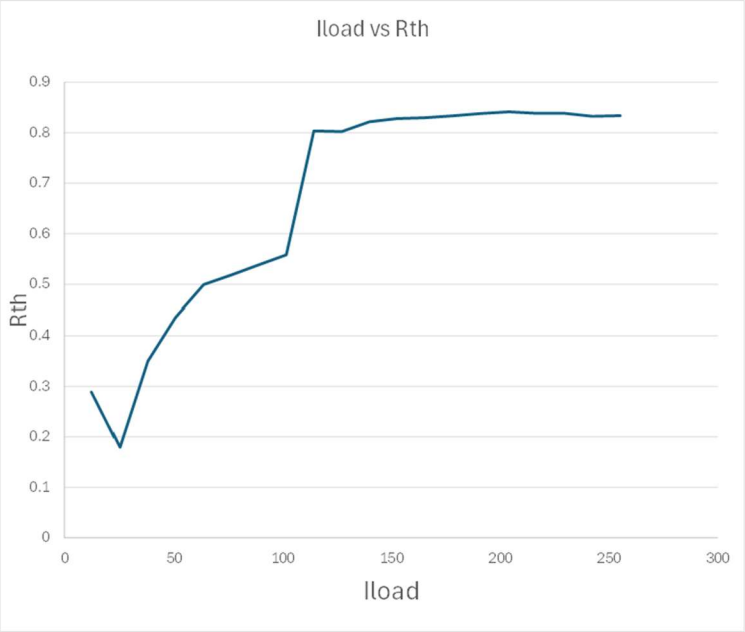
Thevenin's Resistance = $\frac{Vth-Vload}{Iload} = \frac{8.91-8.73}{0.2} = 0.9\ ohms$ when current is 200 mA.

13. THEVENIN'S RESISTANCE OF DIFFERENT POWER SUPPLIES

Different power supplies were chosen and their Thevenin's resistance was measured using instrument droid. The data is collected from Serial monitor and plotted as a graph using excel.

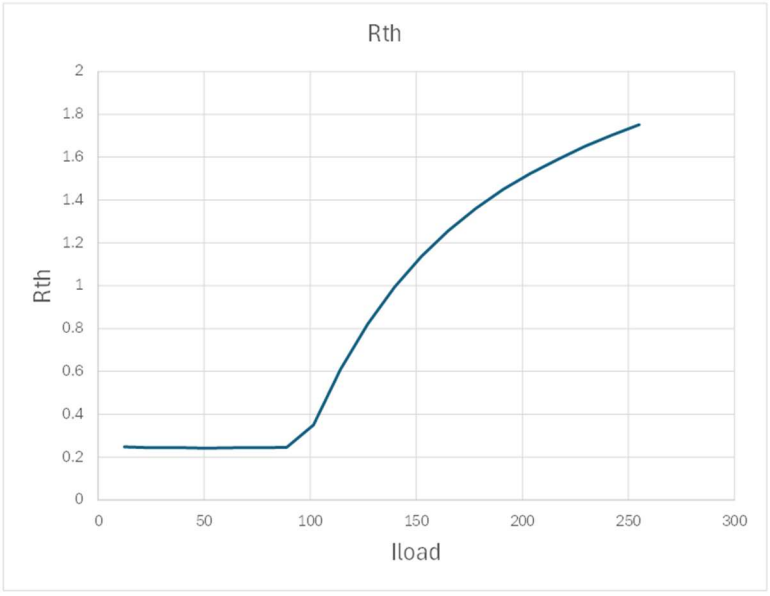
a) 5V POWER ADAPTER : 0.8 OHMS

Index	Iload	Vth	Vload	Rth
1	12.385	5.3527	5.3491	0.2888
2	25.072	5.3526	5.3481	0.1794
3	37.852	5.3529	5.3397	0.3496
4	50.575	5.3531	5.3311	0.4358
5	63.544	5.3531	5.3213	0.4995
6	76.25	5.3532	5.3137	0.5183
7	88.901	5.3527	5.3049	0.5386
8	101.546	5.3528	5.2961	0.5586
9	114.202	5.352	5.2603	0.8036
10	127.082	5.3528	5.2508	0.8023
11	139.826	5.353	5.2381	0.8215
12	152.411	5.353	5.2268	0.8278
13	165.07	5.3526	5.2156	0.8298
14	178.093	5.3528	5.2043	0.8336
15	191.06	5.3528	5.1927	0.8379
16	203.952	5.3534	5.1818	0.8415
17	216.764	5.353	5.1712	0.8387
18	229.458	5.3533	5.161	0.8383
19	242.247	5.3519	5.1502	0.8327
20	255.161	5.3523	5.1395	0.834



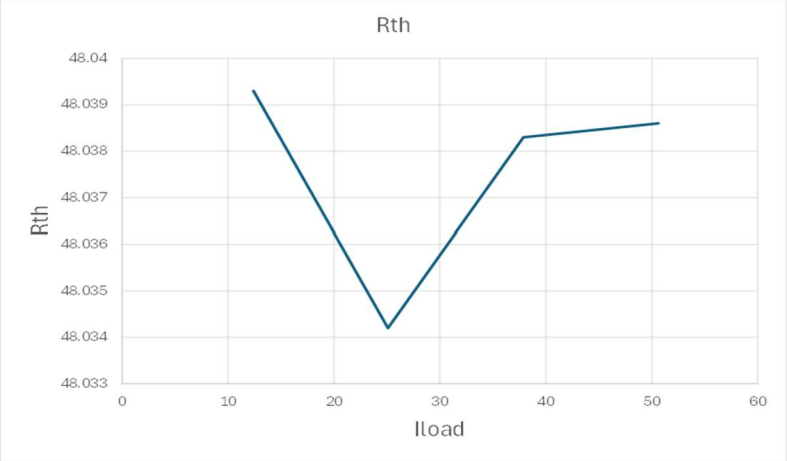
b) 3.3V ARDUINO : 1.7 OHMS

Index	Iload	Vth	Vload	Rth
1	12.388	3.3593	3.3563	0.2477
2	25.068	3.3593	3.3532	0.2433
3	37.839	3.3594	3.3501	0.2452
4	50.563	3.3594	3.3472	0.2412
5	63.526	3.3594	3.3439	0.2441
6	76.235	3.3593	3.3408	0.2438
7	88.88	3.3594	3.3375	0.2462
8	101.527	3.3594	3.3239	0.3499
9	114.175	3.3594	3.2899	0.609
10	127.039	3.3594	3.2552	0.8203
11	139.788	3.3595	3.2205	0.9943
12	152.371	3.3595	3.1864	1.136
13	165.022	3.3595	3.1521	1.2565
14	178.021	3.3596	3.1174	1.3603
15	190.994	3.3595	3.0827	1.4498
16	203.885	3.3596	3.0488	1.5241
17	216.723	3.3596	3.015	1.5898
18	229.398	3.3596	2.9809	1.6509
19	242.192	3.3596	2.947	1.7034
20	255.053	3.3596	2.9128	1.7518



c) FUNCTION GENERATOR : 50 OHMS

Index	Iload	Vth	Vload	Rth
1	12.393	9.604	9.0086	48.0393
2	25.092	9.604	8.3987	48.0342
3	37.875	9.604	7.7845	48.0383
20	50.613	9.604	7.1726	48.0386



14. BEST DESIGN PRACTICES FOLLOWED

1. Decoupling capacitor is placed near the IC to minimize voltage noise due to the voltage drop across the loop inductance in the power rail.
2. Ferrite bead was placed to filter noise for AVCC pin in ATMEGA328 which is sensitive to noise.
3. Proper labelling was given to all switches and test points for better understanding of the board
4. Switches were placed between different sections and proper indicating LEDs were placed to aid for debugging
5. Continuous ground plane was provided on internal layers for proper return path and thicker trace widths were used for power nets.
6. Chose 1206 packages for easy hand assembly.
7. Proper return vias were placed adjacent to signal vias to improve signal integrity.

15. IMPROVEMENTS TO BE IMPLEMENTED

Test points could have been placed apart from each other as it was difficult to probe and measure multiple test point at the same time.

16. CONCLUSIONS

The 'Instrument Droid' was engineered to measure the Thevenin equivalent resistance of power supplies up to 12V. To ensure signal integrity and manufacturability, the PCB design adheres to industry best practices, including continuous return planes, tight decoupling at ICs (with ferrite filtering on AVCC), and standardized 1206 component packages. The layout utilizes 20 mil traces for power and 6 mil for signals, with minimized unique parts and clearly labeled test points to facilitate debugging.

17. APPENDIX

```
// vrm characterizer board
#include <Wire.h>
#include <Adafruit_MCP4725.h>
#include <Adafruit_ADS1X15.h>
#include "pitches.h"
Adafruit_ADS1115 ads;
Adafruit_MCP4725 dac;

//OLED
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
//#include <Fonts/FreeSerif9pt7b.h>
#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);//-1 means no
reset pin on OLED

//LED
#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
#include <avr/power.h>
#endif
#define LEDPIN 9
Adafruit_NeoPixel pixels(4, LEDPIN, NEO_GRB + NEO_KHZ800);
# define buzzer 8

//global variables
float R_sense; //current sensor
long itime_on_msec =100; //on time for taking measurements
long itime_off_msec =itime_on_msec * 10; // time to cool off
int iCounter_off = 0;// counter for number of samples off
int iCounter_on = 0; //counter for number of samples on
float v_divider = 5000.0 / 15000.0; //voltage divider on the VRM
float DAC_ADU_per_v = 4095.0 / 5.0; //conversion from volts to ADU
int V_DAC_ADU; // the value in ADU to output on the DAC
int I_DAC_ADU; // the current we want to output
float I_A = 0.0; //the current we want to output,in amps
long itime_stop_usec; // this is the stop time for each loop
float ADC_V_per_ADU = 0.125 * 1e-3; // the voltage of one bit on the gainof 1
scale
float V_VRM_on_v; // the value of the VRM voltage
float V_VRM_off_v; // the value of the VRM voltage
float I_sense_on_A; // the current through the sense resistor
float I_sense_off_A; // the current through the sense resistor
float I_max_A; // max current to set for
int npts = 20; //number of points to measure
float I_step_A; //step current change
float I_load_A; // the measured current load
float V_VRM_thevenin_v; float V_VRM_loaded_v;
float R_thevenin; int i;
```

```
//The next part is the set up for the ADC and the DAC:
void setup() { Serial.begin(115200);
pinMode(buzzer,OUTPUT);
pixels.begin();
if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
Serial.println("SSD1306 allocation failed");
for(;;);
}

// --- SELECT SENSE RESISTOR HERE ---
bool use1ohm = false;  // set true for 1Ω, false for 10Ω
if (use1ohm) {
  R_sense = 1.0;
  I_max_A = 2.5;
} else {
  R_sense = 10.0;
  I_max_A = 0.25;
}
I_step_A = I_max_A / npts;


display.clearDisplay();
display.setTextSize(2);
display.setTextColor(WHITE);
display.setCursor(1,0);
display.println("GBA Droid");
display.setCursor(0,40);
display.setTextSize(1);
display.println("Measuring RTH");
display.display();
delay(1000);
display.clearDisplay() ;
display.display();
dac.begin(0x60); // address is either 0x60, 0x61, 0x62,0x63, 0x64 or 0x65
dac.setVoltage(0, false); //sets the output current to 0 initially
// ads.setGain(GAIN_TWOTHIRDS); // 2/3x gain +/- 6.144V 1 bit = 3mV 0.1875mV
(default)
ads.setGain(GAIN_ONE); // 1x gain +/- 4.096V 1 bit = 2mV 0.125mV
// ads.setGain(GAIN_TWO); // 2x gain +/- 2.048V 1 bit = 1mV 0.0625mV
// ads.setGain(GAIN_FOUR); // 4x gain +/- 1.024V 1 bit = 0.5mV 0.03125mV
// ads.setGain(GAIN_EIGHT); // 8x gain +/- 0.512V 1 bit = 0.25mV 0.015625mV
// ads.setGain(GAIN_SIXTEEN); // 16x gain +/- 0.256V 1 bit = 0.125mV 0.0078125mV
ads.begin(); // note- you can put the address of the ADS111 here if needed
ads.setDataRate(RATE_ADS1115_860SPS);// sets the ADS1115 for higher speed

// DEBUG: beep once to prove setup() finished
tone(buzzer, 2000, 200);
delay(300);
noTone(buzzer);

}

//loop begins
void loop() {
```

```
    pixels.clear(); // Set all pixel colors to 'off'
    pixels.show();
    noTone(buzzer);
    for (i = 1; i <= npts; i++) {
    display.clearDisplay();
    I_A = i * I_step_A; dac.setVoltage(0, false); //sets the output current
    func_meas_off(); func_meas_on();
    dac.setVoltage(0, false); //sets the output current
    I_load_A = I_sense_on_A - I_sense_off_A; //load current
    V_VRM_thevenin_v = V_VRM_off_v;
    V_VRM_loaded_v = V_VRM_on_v;
    R_thevenin = (V_VRM_thevenin_v - V_VRM_loaded_v) / I_load_A;
    if (V_VRM_loaded_v < 0.75 * V_VRM_thevenin_v) i = npts; //stops the ramping
    Serial.print(i);
    Serial.print(", ");
    display.setCursor(0, 0);
    display.setTextSize(1);
    display.print("index: ");
    display.print(i);
    display.display();
    Serial.print(I_load_A * 1e3, 3);
    Serial.print(", ");
    display.setCursor(0, 9);
    display.setTextSize(1);
    display.print("Iload: ");
    display.print(I_load_A * 1e3);
    display.print("mA");
    display.display();
    Serial.print(V_VRM_thevenin_v, 4);
    Serial.print(", ");
    display.setCursor(0, 20);
    display.setTextSize(1);
    display.print("Vthevenin: ");
    display.print(V_VRM_thevenin_v);
    display.print("V");
    display.display();
    Serial.print(V_VRM_loaded_v, 4);
    Serial.print(", ");
    display.setCursor(0, 30);
    display.setTextSize(1);
    display.print("Vload: ");
    display.print(V_VRM_loaded_v);
    display.print("V");
    display.display();
    Serial.println(R_thevenin, 4);
    display.setCursor(0, 40);
    display.setTextSize(1);
    display.print("Rthevenin: ");
    display.print(R_thevenin);
    display.print("ohms");
    display.display();
    tone(buzzer,1000,50);
    pixels.clear();
    pixels.setPixelColor((i%4), pixels.Color(255, 50, 0));
```

```
    pixels.show();
    }
    Serial.println("done");
    display.setCursor(0, 50);
    display.print("done");
    display.display();
    display.print(", Next read: 10s");
    display.display();


    //reading completed tone
    // delay(1000);
    // tone(buzzer,1200,50);
    // delay(100);
    // tone(buzzer,1400,50);
    // delay(100);
    // tone(buzzer,1800,50);
    // delay(100);
    // noTone(buzzer);

    playCompletedTune();

    pixels.clear();
    pixels.show();
    //delay(2000);
    delay(10000);
    display.clearDisplay();
    }


    //functions

    void func_meas_off(){ dac.setVoltage(0, false);
    //sets the output current
    iCounter_off = 0; //starting the current counter
    V_VRM_off_v = 0.0; //initialize the VRM voltage averager
    I_sense_off_A = 0.0; // initialize the current averager
    itime_stop_usec= micros() + itime_off_msec * 1000; // stop time
    while (micros() <= itime_stop_usec) {
    V_VRM_off_v = ads.readADC_Differential_0_1() * ADC_V_per_ADU / v_divider +
    V_VRM_off_v;
    I_sense_off_A = ads.readADC_Differential_2_3() * ADC_V_per_ADU / R_sense +
    I_sense_off_A;
    iCounter_off++;
    }
    V_VRM_off_v = V_VRM_off_v / iCounter_off;
    I_sense_off_A = I_sense_off_A / iCounter_off;
    // Serial.print(iCounter_off);Serial.print(", ");
    // Serial.print(I_sense_off_A * 1e3, 4); Serial.print(", ");
    // Serial.println(V_VRM_off_v, 4);
    }


    void func_meas_on(){
```

```
    //now turn on the current
    I_DAC_ADU = I_A * R_sense * DAC_ADU_per_v;
    dac.setVoltage(I_DAC_ADU, false); //sets the output current
    iCounter_on = 0;
    V_VRM_on_v = 0.0; //initialize the VRM voltage averager
    I_sense_on_A = 0.00; // initialize the current averager
    itime_stop_usec = micros() + itime_on_msec * 1000; // stop time
    while (micros() <= itime_stop_usec) {
    V_VRM_on_v = ads.readADC_Differential_0_1() * ADC_V_per_ADU / v_divider +
    V_VRM_on_v;
    I_sense_on_A = ads.readADC_Differential_2_3() * ADC_V_per_ADU / R_sense
    +I_sense_on_A;
    iCounter_on++;
    } dac.setVoltage(0, false); //sets the output current to zero
    V_VRM_on_v = V_VRM_on_v / iCounter_on;
    I_sense_on_A = I_sense_on_A / iCounter_on;}

    void playCompletedTune() {
    static const int melody[]    = {
    NOTE_E5, NOTE_E5, REST, NOTE_E5, REST, NOTE_C5, NOTE_E5,
    NOTE_G5, REST, NOTE_G4, REST,
    NOTE_C5, NOTE_G4, REST, NOTE_E4,



    };
    static const int durations[] = {
    8, 8, 8, 8, 8, 8, 8,
    4, 4, 8, 4,
    4, 8, 4, 4,



    }; // quarter notes

    const int size = sizeof(melody) / sizeof(melody[0]);

    for (int i = 0; i < size; i++) {
      int duration = 1000 / durations[i];   // 250 ms
      tone(buzzer, melody[i], duration);
      delay(duration * 1.30);
      noTone(buzzer);
    }

    }
```